Autonomous Grid Navigation: An AI Solution using Reinforcement Learning and Search

Designing and comparing Q-learning and classical search for a warehouse delivery-robot case study

In-Course Assessment (ICA)

CIS4049-N — Artificial Intelligence Foundations

Student: CUI JIANGKUN

Student No: F5743005

Teesside University

July 2026

Contents

1 Introduction and case study

Autonomous mobile robots are moving from research laboratories into everyday commercial settings. Modern distribution centres operated by companies such as Amazon and Ocado already deploy fleets of automated guided vehicles (AGVs) that ferry shelves and parcels across a warehouse floor, and the same navigation problem underlies self-driving delivery pods, cleaning robots and planetary rovers. In every case the core question is the same: given a start location and a destination, how should an intelligent agent choose a sequence of actions that reaches the goal as efficiently as possible while avoiding obstacles? This is a classic Artificial Intelligence (AI) problem, and it is the case study addressed in this report.

The scenario modelled here is a single **warehouse delivery robot** that must travel from its charging depot to a drop-off point across a grid-shaped floor. The floor contains impassable storage racks and bands of *congested* aisles that are slow and therefore expensive to traverse. The objective is not merely to reach the goal, but to reach it by the **cheapest** route, where cost reflects the time or energy spent crossing each cell. This distinction — cheapest rather than shortest — is what makes the problem interesting, because the route with the fewest steps is deliberately not the route with the lowest cost.

The purpose of this ICA is to design an AI solution to this problem and to *critically evaluate* the selection and application of the AI techniques used. Rather than applying a single method, the solution deliberately solves the identical task with **two different families of AI technique** taught in the module and compares them:

1. **Reinforcement Learning (RL)**, specifically tabular **Q-learning**, in which the robot is given no map and must learn an optimal navigation policy purely by trial and error (module Weeks 5–6).
2. **Classical search / planning**, in which the robot is given a model of the environment and computes an optimal route directly using **Breadth-First Search (BFS)**, **Uniform-Cost Search (UCS / Dijkstra)** and **A*** (module Weeks 7–9).

Framing the artefact as a head-to-head comparison is a conscious design decision. It directly mirrors the “Search versus Reinforcement Learning” discussion presented in the Week 8 lecture, it exercises the full technical span of the module, and it turns the artefact into an experiment that produces a genuine, defensible conclusion about *when each technique is appropriate* rather than a single black-box result. The remainder of the report reviews the underlying scientific literature (Section 2), documents the environment and the data structures used to represent it (Section 3), describes the implementation of each technique (Section 4), presents and validates the experimental results (Section 5), reflects on the commercial risks and professional issues raised by deploying such a solution (Section 6), and closes with a critical reflection on limitations, learning and future work (Sections 7–8).

2 Review of the scientific literature

2.1 Problem solving as search

The idea that intelligent behaviour can be framed as *search through a state space* is one of the founding pillars of AI and is developed at length in the standard reference text by Russell and Norvig (2021). A search problem is defined by a set of states, a set of actions with associated costs, a transition (successor) function, an initial state and a goal test; a solution is a sequence of actions transforming the start state into a goal state. The warehouse floor maps onto this formalism naturally: each grid cell is a state, the four compass moves are the actions, entering a cell incurs its terrain cost, and the goal test asks whether the robot has reached the drop-off. The module lectures introduce exactly this "pathing" abstraction, in which the state is the agent's `(row, column)` position and the actions are North, South, East and West.

2.2 Uninformed search

Uninformed (or "blind") search strategies explore the state space systematically without any domain-specific guidance. **Breadth-First Search** expands the shallowest unexpanded node first using a first-in-first-out queue; it is complete and, when every action has equal cost, optimal in the number of steps, but it can consume large amounts of memory. **Depth-First Search** expands the deepest node first using a stack, trading completeness for low memory use. Because BFS treats all steps as equal, it does not minimise *cost* when actions have different weights. The algorithm that repairs this is **Uniform-Cost Search**, which orders its frontier by cumulative path cost `g(n)` using a priority queue and is provably optimal for any non-negative edge costs. UCS is the AI-search formulation of the shortest-path algorithm published by Dijkstra (1959), whose "note on two problems in connexion with graphs" remains one of the most cited results in computer science and still underpins routing in road networks and communication systems today.

2.3 Informed search and heuristics

Uniform-Cost Search is optimal but wasteful: with no sense of where the goal lies, it expands cost contours in every direction. **Informed search** adds a *heuristic* function `h(n)` that estimates the remaining cost from a node to the goal, biasing the search towards promising regions. Greedy best-first search follows the heuristic alone and can be badly misled, whereas **A*** search combines both quantities, ordering the frontier by `f(n) = g(n) + h(n)`, the estimated total cost of a path through `n`. The algorithm was introduced by Hart, Nilsson and Raphael (1968), who proved that A* is optimal provided the heuristic is *admissible* — that is, it never overestimates the true remaining cost. For grid navigation the Manhattan distance is the natural admissible heuristic, because when the cheapest possible step costs one unit, the straight-line

grid distance can never exceed the real cost to the goal. Admissibility is precisely what guarantees that A* returns an optimal path while expanding far fewer nodes than UCS, and designing good admissible heuristics — often by relaxing the original problem — is, as Russell and Norvig (2021) note, most of the art of applying A* in practice.

2.4 Reinforcement learning and Q-learning

Search assumes the agent already possesses a perfect model of the environment. Reinforcement learning removes that assumption: an agent learns to act by interacting with an environment, receiving a scalar reward signal, and adjusting its behaviour to maximise cumulative reward over time. The problem is formalised as a Markov Decision Process, and the agent's objective is an optimal *policy* mapping states to actions. Many RL methods work by estimating a *value function*; the action-value or **Q-function** `Q(s, a)` gives the expected cumulative reward of taking action `a` in state `s` and behaving optimally thereafter, and satisfies the Bellman optimality equation.

Q-learning, introduced by Watkins and Dayan (1992), is the landmark algorithm that learns this Q-function directly from experience. After each observed transition `(s, a, r, s')` it applies the temporal-difference update

$$Q(s,a) \leftarrow Q(s,a) + \alpha \left[r + \gamma \max_{a'} Q(s',a') - Q(s,a) \right],$$

where α is the learning rate and γ the discount factor. Crucially, the update uses the *maximum* Q-value available in the next state regardless of the action actually taken next, which makes Q-learning an **off-policy** method: it can learn the optimal policy while behaving more exploratively. Watkins and Dayan proved that, under mild conditions, the estimates converge to the optimal Q-function. The comprehensive modern treatment of these ideas is the textbook by Sutton and Barto (2018), which also contrasts Q-learning with its on-policy sibling **SARSA**, whose update depends on the next action actually chosen. A central practical concern in all of RL is the **exploration–exploitation trade-off**: the agent must try unfamiliar actions often enough to discover their value, yet exploit what it already knows to accumulate reward. The standard remedy, used in this solution, is an *ε-greedy* policy in which a random action is taken with probability ε and ε is gradually annealed towards zero.

2.5 From tabular methods to deep reinforcement learning

Tabular Q-learning stores one value per state–action pair, which becomes infeasible as the state space grows — the "curse of dimensionality" noted in the lectures. The influential work of Mnih et al. (2015) addressed this by approximating the Q-function with a deep neural network, producing the Deep Q-Network (DQN) that learned to

play dozens of Atari games at human level directly from pixels. DQN is conceptually the same Q-learning update studied here, augmented with a function approximator plus stabilising techniques such as experience replay and a target network. This lineage — from Dijkstra's shortest paths, through A* and tabular Q-learning, to deep RL — situates the modest artefact in this report firmly within the mainstream of AI research and signals the natural direction in which it would be scaled.

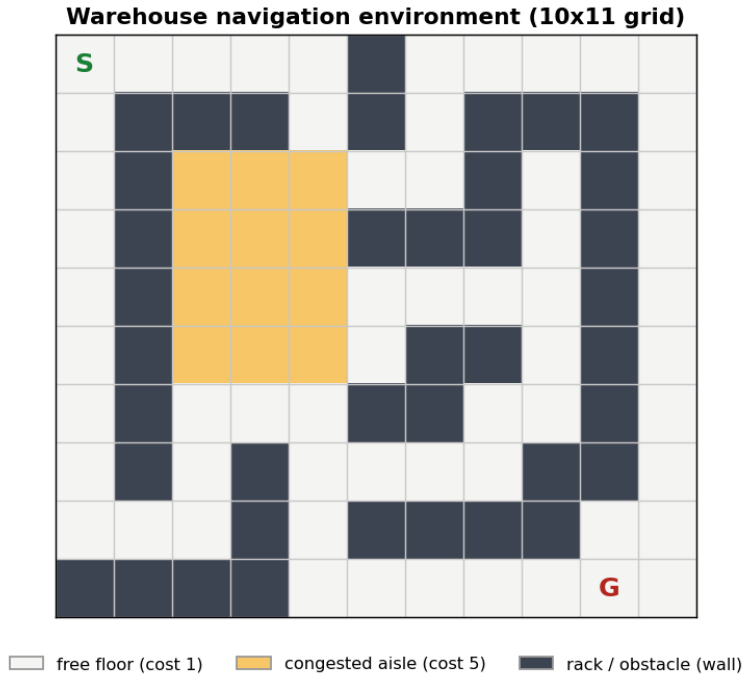
2.6 How the literature shapes the solution

Taken together, the literature motivates the design directly. Search and RL are presented in the lectures as complementary paradigms: search is a systematic, model-based technique for finding solutions in a known space, whereas RL is a model-free learning process for environments that must be discovered through interaction. Because the warehouse task can be posed both ways, it is an ideal vehicle for comparing them, and the classical results above give clear, testable predictions — BFS should minimise steps but not cost, UCS and A* should agree on the optimal cost with A* expanding fewer nodes, and Q-learning should, given enough experience, converge to that same optimal route. Section 5 tests each prediction empirically.

3 The AI solution: environment and data structures

3.1 Environment representation

The environment is implemented from scratch in Python, with no reliance on `gym`/`gymnasium`, so that the artefact is fully transparent and reproducible offline. The warehouse floor is stored as a list of equal-length strings — a compact, human-readable grid in which each character encodes one cell:



A free-floor cell (`.`) costs one unit to enter, a congested aisle (`~`) costs five, and a rack (`#`) is an impassable wall. The start (`S`) and goal (`G`) are located in opposite corners. The 10×11 layout contains 59 free cells, 12 congested cells and 39 wall cells, and it is designed with a specific property: the direct corridor from start to goal passes straight through the congested band, whereas a detour around the free-floor perimeter is longer in steps but cheaper overall. Without this tension the different algorithms would be indistinguishable, so the environment itself is part of the experimental design.

3.2 State space, action space and dynamics

The state is the robot's `(row, column)` position, giving 110 nominal states of which 71 are reachable (non-wall) cells. The action space is the four moves North, South, East and West. Transitions are **deterministic**: an action moves the robot to the adjacent cell unless that cell is a wall or lies outside the grid, in which case the robot remains in place — the identical "infeasible action" rule used by the Taxi environment in the Week 6 lecture. Determinism is a modelling choice that keeps the problem tractable and, importantly, allows the RL result to be cross-checked against the exact optimum computed by search.

3.3 Reward design

For the reinforcement-learning formulation the environment also exposes a reward signal, engineered by *reward shaping* as recommended in the lectures. Entering an

ordinary cell yields a negative reward equal to that cell's terrain cost, which pushes the agent towards short, cheap routes; reaching the goal yields a large positive bonus (`+50`); and attempting an illegal move into a wall incurs a penalty (`−10`) while leaving the robot in place, discouraging wasted actions. Because each move's reward is the negative of its search cost, the reward-maximising policy and the minimum-cost path coincide by construction — which is exactly why the two paradigms can be expected to agree.

3.4 Data structures for the algorithms

Each algorithm uses the data structure that its theory prescribes, and choosing them correctly is itself part of applying the technique well. The Q-learning agent stores its Q-table as a nested dictionary mapping every cell to a dictionary of four action-values, giving direct `O(1)` lookup and update. BFS maintains its frontier as a `collections.deque` used as a first-in-first-out queue; UCS and A* use a binary heap (`heapq`) as a priority queue, keyed by path cost `g(n)` and by `f(n)=g(n)+h(n)` respectively; and all three searches keep a `came_from` dictionary so the final path can be reconstructed by walking backwards from the goal. A visited/closed set prevents the exponential rework that, as the lectures warn, results from re-expanding states in graph search.

4 Implementation of the AI techniques

4.1 Q-learning

The learning agent runs for 1,500 episodes. Each episode resets the robot to the depot and lets it act for at most 200 steps. At every step an action is chosen ε-greedily, the environment returns the next state, reward and termination flag, and the Q-table is updated with the off-policy temporal-difference rule of Section 2.4 using a learning rate `α = 0.1` and discount factor `γ = 0.95`. Exploration is annealed geometrically from `ε = 1.0` down to a floor of `0.05`, so the agent explores aggressively at first and increasingly exploits its improving value estimates. The greedy policy is then read off by taking, in each cell, the action with the highest learned Q-value.

4.2 Search algorithms

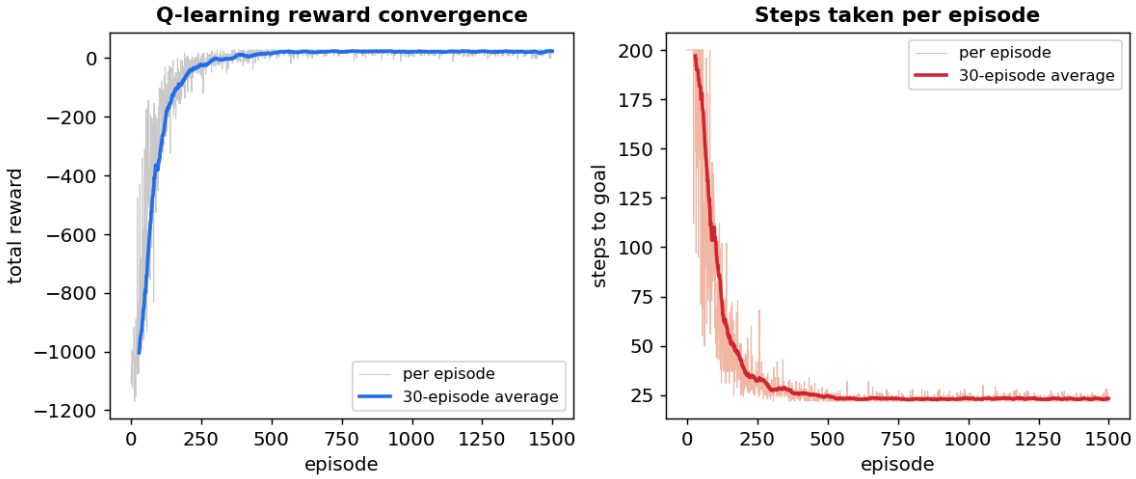
The three search procedures share a common skeleton and differ only in how the frontier is ordered, exactly as the "one queue" observation in the Week 9 lecture predicts. BFS pops from the left of a FIFO queue; UCS pops the lowest-cost node from a priority queue and relaxes neighbours in Dijkstra fashion; A* is identical to UCS except that frontier priority is `g(n)` plus the Manhattan-distance heuristic `h(n)`. The heuristic is admissible because it counts grid steps and the cheapest possible step costs one unit, so it can never overestimate true remaining cost; this is verified

numerically in the artefact. Each search returns the path, its total cost, its length in steps and the number of nodes it expanded, the last of these being the key measure of computational effort.

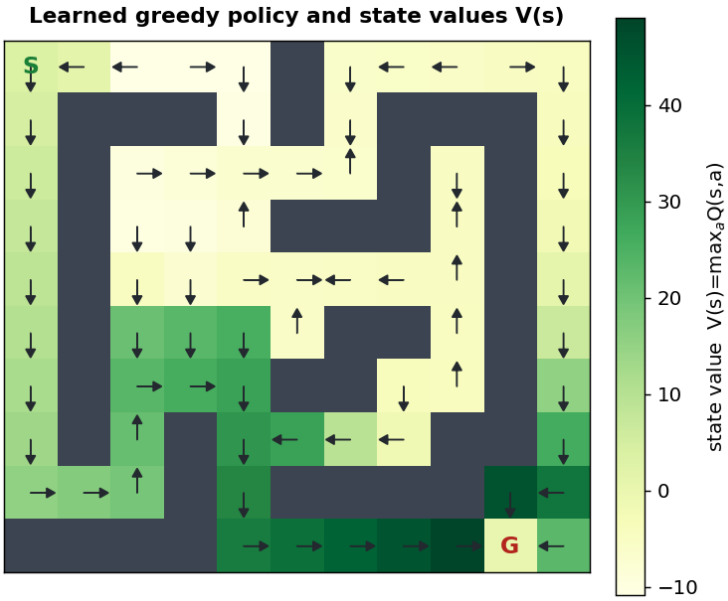
5 Experimental validation and results

5.1 Learning convergence

The Q-learning agent's progress is shown below. Early episodes accumulate large negative reward because the untrained agent wanders for the full step budget; as value information propagates back from the goal, the reward climbs steeply and the number of steps per episode collapses from around 200 to roughly 22, after which both curves plateau — the visual signature of a converged policy.



Reading the greedy action in every cell yields the learned policy, plotted below over the state-value function `V(s) = max_a Q(s,a)`. The value surface rises smoothly towards the goal and the policy arrows form a coherent route from `S` to `G`, confirming that the agent has learned a sensible global plan rather than merely memorising one trajectory.

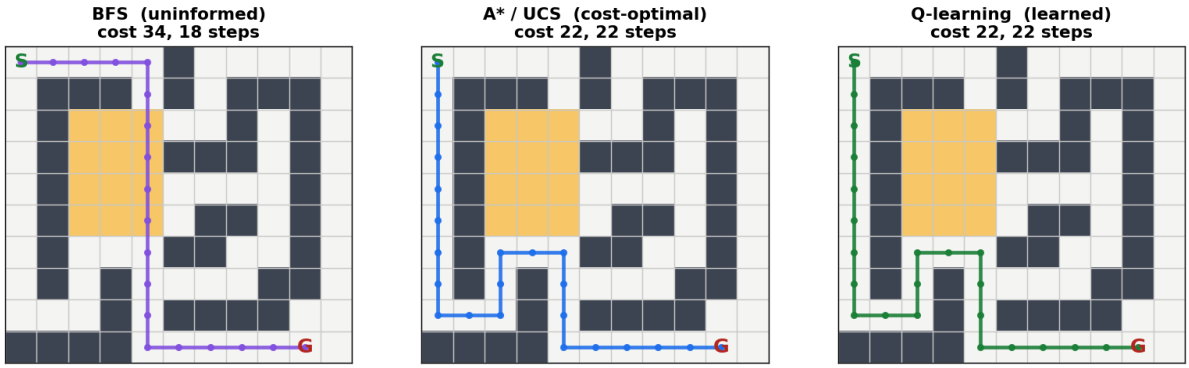


5.2 Search results and the routes chosen

Running the three search algorithms on the known map produces the following headline figures:

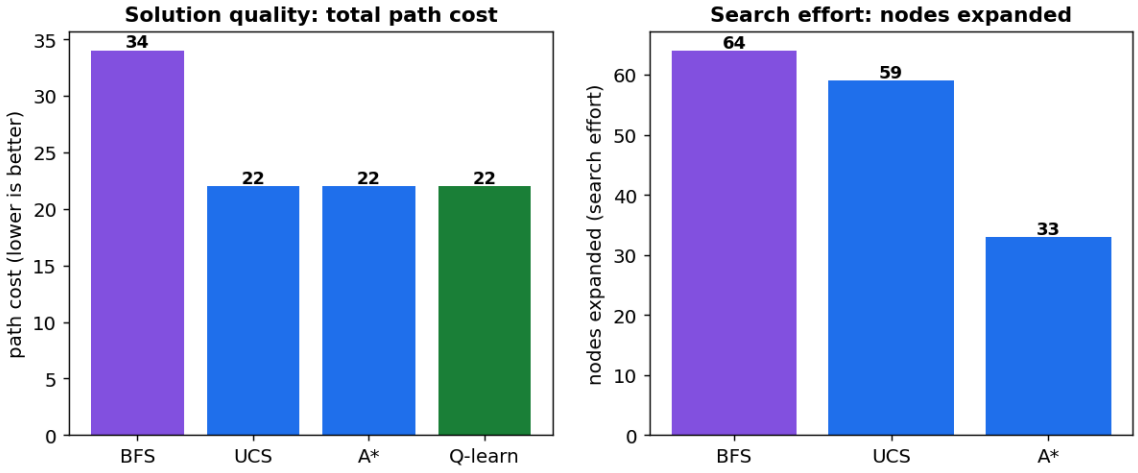
Method	Paradigm	Path cost	Steps	Nodes expanded	Cost-optimal
BFS (uninformed)	Search	34	18	64	No
UCS / Dijkstra	Search	22	22	59	Yes
A* (informed)	Search	22	22	33	Yes
Q-learning (RL)	Learning	22	22	—	Yes*

The routes themselves make the trade-off vivid. BFS drives straight down through the congested band — the fewest steps (18) but the highest cost (34). A*/UCS and the learned Q-learning policy all detour around the free-floor perimeter, taking more steps (22) but a substantially lower cost (22).



5.3 Critical comparison: Search versus Reinforcement Learning

The comparison charts below summarise the two dimensions that matter — solution quality and computational effort.



Four conclusions follow, each of which is asserted programmatically in the artefact so that the notebook fails loudly if the implementation is ever broken. First, **BFS minimises the wrong objective**: it returns the shortest route in steps but a path 55% more expensive than optimal, a concrete illustration of why cost-sensitive search matters whenever actions are not equally cheap. Second, **A* and UCS agree exactly on the optimal cost of 22**, empirically confirming the optimality guarantee of an admissible heuristic. Third, **A* is markedly more efficient than UCS**, expanding only 33 nodes against 59 — very nearly halving the work — because the heuristic focuses the search towards the goal rather than spreading equally in all directions. Fourth, and most striking, **Q-learning independently discovers the same optimal-cost route with no map at all**, learning it from reward feedback alone; the agreement between a model-based planner and a model-free learner cross-validates both implementations.

The deeper lesson is about *assumptions and their price*. Search is exact and fast, but

only because it is handed a perfect model of the warehouse; its cost is measured in nodes expanded, and a good heuristic buys a large reduction in that cost. Reinforcement learning demands no model, which is exactly what makes it applicable when the environment is unknown, stochastic or changing — but it pays for that generality in *sample cost* (roughly 1,500 episodes of trial and error) and in the weaker nature of its guarantee: its optimality here is empirical, and states far from the optimal route remain under-explored, so their learned values are noisier and less trustworthy. This is precisely the "Search versus RL" position set out in the module: **use search when you have a reliable model of the world, and reinforcement learning when you must learn the world by interacting with it.**

6 Commercial risks and professional issues

Translating this artefact into a deployed warehouse system raises real commercial and professional considerations that any responsible engineer must weigh.

Commercial risks. The most immediate risk is the gap between simulation and reality. The clean, deterministic grid used here is a convenient abstraction; a real floor has continuous geometry, sensor noise, wheel slip, moving people and other robots, and none of these are captured. A policy that is optimal in simulation can fail expensively when transferred to hardware — the well-known *sim-to-real* problem — so budget and schedule must allow for extensive on-site testing. Scalability is a second commercial risk: tabular Q-learning stores a value for every state, so memory and training time grow with the size and complexity of the floor, and a naive deployment across a large multi-aisle facility would quickly become intractable without the function-approximation methods of Mnih et al. (2015). Third, the objective actually encoded must match the business objective: optimising purely for path cost ignores throughput, battery scheduling, congestion between many robots and deadlines, and a system that is locally optimal per robot can be globally inefficient for the fleet. Finally, there is a maintenance cost: warehouses are re-planogrammed frequently, and each change invalidates a learned policy or a cached plan, so the total cost of ownership includes continual retraining and revalidation, not just the initial build.

Professional and ethical issues. Autonomous vehicles operating around people are safety-critical systems, and this brings professional obligations that the British Computer Society and IEEE codes of conduct make explicit. Safety must be demonstrable: a learned policy is comparatively opaque, and it is harder to certify or to explain after an incident than a deterministic search plan whose every decision can be audited — an argument, in safety-critical settings, for using search (or a verifiable planner constrained by hard safety rules) rather than an unconstrained learned controller. Accountability and liability for the actions of an autonomous robot are unresolved questions that a deploying organisation must confront contractually and

legally. There are also workforce implications: automation of materials handling displaces manual roles, and professionalism requires honest engagement with that impact rather than treating it as an externality. Reproducibility and transparency are professional virtues too, which is why this artefact fixes its random seed, documents its assumptions, and validates its results with explicit checks. Where robots gather sensor or camera data about a workplace, data-protection and worker-surveillance concerns arise and must be handled under the applicable regulations. None of these issues is a reason not to build such systems, but each is a reason to build them carefully, transparently and with appropriate human oversight.

7 Reflection, limitations and future work

What I learned and the challenges I met. The module took me from a general idea of "AI" to a working understanding of two concrete families of technique and, more importantly, of *when to choose which*. Building the artefact rather than merely reading about it exposed subtleties that the lectures foreshadowed. Designing the environment was harder than expected: my first layouts made the shortest and cheapest routes identical, so every algorithm agreed and the comparison said nothing — I had to engineer the congested band deliberately to create a meaningful trade-off. Reward shaping required care, because an over-generous goal bonus or an under-weighted step penalty produced policies that reached the goal but not by the cheapest route. Tuning Q-learning was an exercise in the exploration–exploitation balance: too little exploration and the agent locked onto a poor route; too fast an ε decay and it never discovered the perimeter detour. Verifying that my Manhattan heuristic was genuinely admissible, and checking the learned policy against the exact search optimum, taught me to treat validation as part of the implementation rather than an afterthought.

Limitations. The solution is deliberately small and its limits should be stated plainly. The environment is deterministic and static, whereas real warehouses are stochastic and dynamic. The state space is tiny and fully enumerable, so tabular Q-learning suffices; it would not scale. Only a single start–goal pair is considered, and only one obstacle layout. The RL agent's optimality is empirical, established for this problem by comparison with search, not guaranteed in general.

Future work. Several extensions follow naturally and map onto the literature reviewed above. Introducing stochastic transitions (a probability that the robot slips sideways) would make the problem genuinely uncertain and would let me compare off-policy Q-learning against on-policy SARSA, whose more cautious behaviour is known to differ near hazards. Replacing the Q-table with a neural network would move the solution towards the DQN of Mnih et al. (2015) and allow much larger floors. A systematic hyperparameter grid search over α, γ and the ε schedule — exactly the optimisation the lecturer suggested as an exercise — would sharpen convergence.

Finally, a hybrid architecture that plans with A* over a known static map but hands control to a learned policy when the environment departs from that map would combine the exactness of search with the adaptivity of RL, and is the direction I would pursue in a larger project and, potentially, in future study or employment in robotics and autonomous systems.

8 Conclusion

This report designed and implemented an AI solution to a warehouse-navigation case study and used it to compare two central paradigms of the module. Classical search — BFS, Uniform-Cost Search / Dijkstra, and A* — solves the task exactly when a model of the environment is available, with A* reaching the optimal-cost route while expanding roughly half the nodes of uninformed search thanks to an admissible heuristic. Tabular Q-learning solves the identical task with no model, learning the same optimal route from reward feedback alone at the price of sample cost and weaker guarantees. Their agreement on the optimal cost of 22 cross-validates both implementations, and their differing assumptions yield a clear, defensible engineering conclusion: plan with search when the world is known, and learn with reinforcement learning when it is not. The exercise met all of the module's learning outcomes and, beyond the marks, left me with a working grasp of AI problem-solving that I expect to build on.

References

Dijkstra, E. W. (1959) 'A note on two problems in connexion with graphs', *Numerische Mathematik*, 1, pp. 269–271.

Hart, P. E., Nilsson, N. J. and Raphael, B. (1968) 'A formal basis for the heuristic determination of minimum cost paths', *IEEE Transactions on Systems Science and Cybernetics*, 4(2), pp. 100–107.

Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A. A., Veness, J., Bellemare, M. G., Graves, A., Riedmiller, M., Fidjeland, A. K., Ostrovski, G. et al. (2015) 'Human-level control through deep reinforcement learning', *Nature*, 518(7540), pp. 529–533.

Russell, S. J. and Norvig, P. (2021) *Artificial Intelligence: A Modern Approach*. 4th edn. Hoboken, NJ: Pearson.

Sutton, R. S. and Barto, A. G. (2018) *Reinforcement Learning: An Introduction*. 2nd edn. Cambridge, MA: MIT Press.

Watkins, C. J. C. H. and Dayan, P. (1992) 'Q-learning', *Machine Learning*, 8(3–4), pp. 279–292.